# Islamic University of Technology

Organization of Islamic Cooperation




| | |
|---|---|
| **Name** | **:** Antara Arifa Mullick |
| **ID** | **:** 210042132 |
| **Program** | **:** SWE |
| **Department** | **:** CSE |
| **Subject** | **:** Artificial Intelligence |
| **Subject Code** | **:** CSE 4618 |
| **Title** | **:** Informed Search |
| **Date of Submission** | **:** 12th June, 2025 |

# 1. Introduction

This lab focused on implementing informed search algorithms, specifically A* search, and applying them to various Pacman navigation problems. The primary objectives were to:

- Implement A* graph search algorithm with heuristic functions.
- Design custom search problems for corner navigation and food collection.
- Develop admissible and consistent heuristics for complex state spaces.
- Compare performance between different search strategies.

The lab demonstrated the power of informed search over uninformed search by incorporating domain knowledge through heuristic functions, significantly reducing the search space exploration.

# 2. Problem Analysis

## 2.1 Core Challenges

The lab presented several interconnected challenges:

1. **State Space Design**: Choosing appropriate state representations that capture essential information while avoiding irrelevant details.
2. **Heuristic Design**: Creating admissible and consistent heuristics that provide good guidance without overestimating costs.
3. **Search Efficiency**: Balancing optimality with computational efficiency.
4. **Problem Formulation**: Translating complex scenarios (like visiting all corners) into formal search problems.

## 2.2 Key Algorithmic Concepts

- **A Search*:** Combines actual cost (g) with heuristic estimate (h) using f = g + h.
- **Admissibility**: Heuristic never overestimates the true cost to goal.
- **Consistency**: For any node n and successor n', $h(n) \leq c(n,n') + h(n')$.
- **Graph Search**: Maintains visited set to avoid revisiting states.

# 3. Solution Explanation

## 3.1 Question 1: A* Search Implementation

```
def aStarSearch(problem, heuristic=nullHeuristic):
    fringe = util.PriorityQueue()
    visited = []
    start = problem.getStartState()
    fringe.push((start, []), heuristic(start, problem))

    while not fringe.isEmpty():
        currState, actionList = fringe.pop()
        if problem.isGoalState(currState):
            return actionList

        if currState not in visited:
            visited.append(currState)
                                    for   nextState,   action,   cost   in
problem.getSuccessors(currState):
                newActions = actionList + [action]
                g = problem.getCostOfActions(newActions)
                h = heuristic(nextState, problem)
                fringe.push((nextState, newActions), g + h)

    return []
```

**Key Design Decisions:**

- Used PriorityQueue with f-value (g + h) as priority.
- Maintained visited list for graph search to avoid cycles.
- Calculated g-cost using `getCostOfActions` for accuracy.
- Combined path cost with heuristic estimate for informed search.

## 3.2 Question 2: Corners Problem

**State Representation:**

```
self.startState = (self.startingPosition, (False, False, False, False))
```

The state consists of:

- Current position: (x, y) coordinates.
- Visited corners: 4-tuple of boolean values.

**Successor Generation:**

```
visited = list(visitedCorners)
for idx, corner in enumerate(self.corners):
    if nextPosition == corner:
        visited[idx] = True
successors.append(((nextPosition, tuple(visited)), action, 1))
```

This approach efficiently tracks which corners have been visited while maintaining the search state's immutability.

## 3.3 Question 3: Corners Heuristic

```
def cornersHeuristic(state, problem):
    corners = problem.corners
    current_position = state[0]
    visited_corners = state[1]

    unvisited = [corner for idx, corner in enumerate(corners)
                 if not visited_corners[idx]]

    heuristic = 0
    position = current_position
    unvisited_corners = unvisited.copy()

    while unvisited_corners:
          distances = [abs(position[0] - corner[0]) + abs(position[1] -
corner[1])
                       for corner in unvisited_corners]
        min_distance = min(distances)
        heuristic += min_distance

        min_index = distances.index(min_distance)
        position = unvisited_corners[min_index]
        unvisited_corners.pop(min_index)

    return heuristic
```

**Heuristic Strategy:**

- Uses greedy nearest-neighbor approach.
- Computes Manhattan distance to closest unvisited corner.
- Iteratively "visits" closest corners to estimate remaining cost.
- Guarantees admissibility as it uses straight-line distances.

## 3.4 Question 4: Food Search Heuristic

```
def foodHeuristic(state, problem):
    position, foodGrid = state
    foodList = foodGrid.asList()
    if not foodList:
        return 0

    # Minimum distance from Pacman to any food
    minPacmanFood = min([abs(position[0] - food[0]) + abs(position[1] - food[1])
                         for food in foodList])

    # Maximum distance between any two food dots
    foodDiameter = 0
    for i in range(len(foodList)):
        for j in range(i+1, len(foodList)):
             dist = abs(foodList[i][0] - foodList[j][0]) + abs(foodList[i][1] -
foodList[j][1])
            if dist > foodDiameter:
                foodDiameter = dist

    return minPacmanFood + foodDiameter
```

**Heuristic Components:**

1. **Minimum Pacman-to-Food Distance**: Ensures we account for reaching the food cluster.
2. **Food Diameter**: Estimates the cost of traversing the food distribution.
3. **Combination**: Provides a lower bound on the total collection cost.

### 3.5 Question 5: Closest Dot Search

```
def findPathToClosestDot(self, gameState):
    problem = AnyFoodSearchProblem(gameState)
    return search.bfs(problem)

def isGoalState(self, state):
    x, y = state
    return self.food[x][y]
```

**Strategy:**

- Uses BFS to find the shortest path to any food dot.
- Greedy approach: always go to the nearest food.
- Suboptimal but computationally efficient.

# 4. Performance Analysis

### 4.1 Results Summary

| Question | Score | Key Metrics |
|---|---|---|
| Q1 | 3/3 | A* implementation passed all test cases |
| Q2 | 3/3 | Corners problem solved optimally (28 steps) |
| Q3 | 3/3 | Heuristic expanded 692 nodes (< 1200 threshold) |
| Q4 | 4/4 | Food heuristic within efficiency bounds |
| Q5 | 3/3 | Closest dot search implemented correctly |

### 4.2 Complexity Analysis

- **A Time Complexity*:** $O(b^d)$ where b is the branching factor, d is depth.
- **Space Complexity**: $O(b^d)$ for storing frontier and visited states.
- **Corners Heuristic**: $O(k^2)$where k is the number of unvisited corners.
- **Food Heuristic**: $O(n^2)$ where n is the number of remaining food dots.

# 5. Findings and Insights

## 5.1 Heuristic Design Insights

1. **Admissibility vs. Informativeness Trade-off**: More informative heuristics risk becoming inadmissible
2. **Problem Structure Matters**: Effective heuristics exploit problem-specific structure
3. **Consistency Importance**: Consistent heuristics guarantee optimal solutions in graph search

## 5.2 Performance Observations

1. **A vs UCS*:** A* with good heuristics significantly reduces node expansion.
2. **Heuristic Quality Impact**: Better heuristics lead to more focused search.
3. **State Representation**: Compact state representation improves memory efficiency.

## 5.3 Algorithm Behavior

- **Corners Problem**: Greedy nearest-corner heuristic provides good guidance
- **Food Problem**: Diameter-based heuristic captures distribution geometry
- **Suboptimal Search**: Greedy closest-dot approach trades optimality for speed

# 6. Challenges Faced and Solutions

## 6.1 State Representation Challenge

**Problem**: Choosing appropriate state representation for corners problem **Solution**: Used tuple combining position and visited-corners boolean array.

## 6.2 Heuristic Consistency

**Problem**: Ensuring heuristics remain consistent while being informative **Solution**: Used conservative distance-based estimates with mathematical proofs.

## 6.3 Efficiency vs. Optimality

**Problem**: Balancing search efficiency with solution quality **Solution**: Designed heuristics that provide good guidance without overestimation.

# 7. Hyperparameter Exploration

## 7.1 Heuristic Function Variants

Tested different approaches for corners heuristic:

- **Simple Manhattan**: Distance to furthest corner.
- **Greedy MST**: Minimum spanning tree approximation.
- **Nearest Neighbor**: Implemented greedy approach (chosen for balance).

# 8. Additional Insights

## 8.1 Problem-Specific Optimizations

- **Corners**: Exploited geometric structure of corner positions.
- **Food**: Leveraged distribution characteristics of food placement.
- **Closest Dot**: Utilized locality principle for greedy decisions.

## 8.2 Real-World Applications

The techniques learned apply to:

- **Robotics**: Path planning with obstacles.
- **Game AI**: NPC navigation and strategy.
- **Logistics**: Route optimization with constraints.

# 9. Conclusion

This lab successfully demonstrated the power of informed search through A* algorithm implementation and heuristic design. Key achievements include:

1. **Perfect Implementation**: All algorithms passed comprehensive test suites.
2. **Efficient Heuristics**: Designed admissible and consistent heuristics that significantly improved search performance.
3. **Problem Formulation**: Successfully translated complex navigation problems into formal search frameworks.

4. **Performance Optimization**: Achieved efficient solutions through careful state representation and heuristic design.

The experience reinforced the importance of domain knowledge in AI problem-solving and the delicate balance between computational efficiency and solution optimality. The lab provided practical experience with fundamental AI search techniques that form the backbone of many intelligent systems.